Python Variable Scope

The **scope of a variable** in Python is defined as the specific area or region where the variable is accessible to the user. The scope of a variable depends on where and how it is defined. In Python, a variable can have either a global or a local scope.

Types of Scope for Variables in Python

On the basis of scope, the Python variables are classified in three categories –

- Local Variables
- Global Variables
- Nonlocal Variables

Local Variables

A local variable is defined within a specific function or block of code. It can only be accessed by the function or block where it was defined, and it has a limited scope. In other words, the scope of local variables is limited to the function they are defined in and attempting to access them outside of this function will result in an error. Always remember, multiple local variables can exist with the same name.

Example

The following example shows the scope of local variables.

```
def myfunction():
   a = 10
   b = 20
   print("variable a:", a)
   print("variable b:", b)
   return a+b

print (myfunction())
```

In the above code, we have accessed the local variables through its function. Hence, the code will produce the following output –

```
variable a: 10
variable b: 20
30
```

Global Variables

A global variable can be accessed from any part of the program, and it is defined outside any function or block of code. It is not specific to any block or function.

Example

The following example shows the scope of global variable. We can access them inside as well as outside of the function scope.

```
#global variables
name = 'TutorialsPoint'
marks = 50
def myfunction():
   # accessing inside the function
   print("name:", name)
   print("marks:", marks)
# function call
myfunction()
```

The above code will produce the following output –

```
name: TutorialsPoint
marks: 50
```

Nonlocal Variables

The Python variables that are not defined in either local or global scope are called nonlocal variables. They are used in nested functions.

Example

The following example demonstrates the how nonlocal variables works.

```
def yourfunction():
   a = 5
   b = 6
   # nested function
   def myfunction():
      # nonlocal function
      nonlocal a
      nonlocal b
      a = 10
      b = 20
      print("variable a:", a)
      print("variable b:", b)
      return a+b
   print (myfunction())
yourfunction()
```

The above code will produce the below output –

```
variable a: 10
variable b: 20
30
```

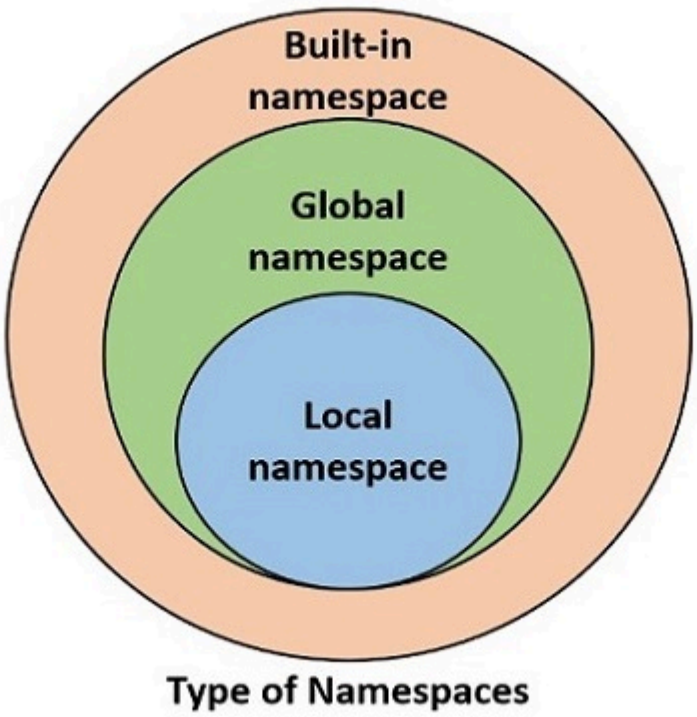
Namespace and Scope of Python Variables

A namespace is a collection of identifiers, such as variable names, function names, class names, etc. In Python, namespace is used to manage the scope of variables and to prevent naming conflicts.

Python provides the following types of namespaces –

- **Built-in namespace** contains built-in functions and built-in exceptions. They are loaded in the memory as soon as Python interpreter is loaded and remain till the interpreter is running.
- **Global namespace** contains any names defined in the main program. These names remain in memory till the program is running.
- **Local namespace** contains names defined inside a function. They are available till the function is running.

These namespaces are nested one inside the other. Following diagram shows relationship between namespaces.



Type of Namespaces

The life of a certain variable is restricted to the namespace in which it is defined. As a result, it is not possible to access a variable present in the inner namespace from any outer namespace.

Python globals() Function

Python's standard library includes a built-in function **`globals()`**. It returns a dictionary of symbols currently available in global namespace.

Run the globals() function directly from the Python prompt.

```
>>> globals()
{'__name__': '__main__', '__doc__': None, '__package__': None, '__loader__': <cla
```

It can be seen that the built-in module which contains definitions of all built-in functions and built-in exceptions is loaded.

Example

Save the following code that contains few variables and a function with few more variables inside it.

```
name = 'TutorialsPoint'
marks = 50
result = True
def myfunction():
   a = 10
   b = 20
   return a+b

print (globals())
```

Calling globals() from inside this script returns following dictionary object –

```
{'__name__': '__main__', '__doc__': None, '__package__': None, '__loader__': <_fr
```

The global namespace now contains variables in the program and their values and the function object in it (and not the variables in the function).

Python locals() Function

Python's standard library includes a built-in function **`locals()`**. It returns a dictionary of symbols currently available in the local namespace of the function.

Example

Modify the above script to print dictionary of global and local namespaces from within the function.

```
name = 'TutorialsPoint'
marks = 50
result = True
def myfunction():
   a = 10
   b = 20
   c = a+b
   print ("globals():", globals())
   print ("locals():", locals())
   return c
myfunction()
```

The output shows that locals() returns a dictionary of variables and their values currently available in the function.

```
globals(): {'__name__': '__main__', '__doc__': None, '__package__': None, '__loac
locals(): {'a': 10, 'b': 20, 'c': 30}
```

Since both globals() and locals functions return dictionary, you can access value of a variable from respective namespace with dictionary get() method or index operator.

```
print (globals()['name']) # displays TutorialsPoint
print (locals().get('a')) # displays 10
```

Namespace Conflict in Python

If a variable of same name is present in global as well as local scope, Python interpreter gives priority to the one in local namespace.

Example

In the following example, we define a local and a global variable.

```
marks = 50 # this is a global variable
def myfunction():
   marks = 70 # this is a local variable
   print (marks)

myfunction()
print (marks) # prints global value
```

It will produce the following output –

```
70
50
```

Example

If you try to manipulate value of a global variable from inside a function, Python raises **UnboundLocalError** as shown in example below –

```
# this is a global variable
marks = 50
def myfunction():
   marks = marks + 20
   print (marks)

myfunction()
# prints global value
print (marks)
```

It will produce the following error message –

```
   marks = marks + 20
           ^^^^^
UnboundLocalError: cannot access local variable 'marks' where it is not associate
```

Example

To modify a global variable, you can either update it with a dictionary syntax, or use the **global** keyword to refer it before modifying.

```
var1 = 50 # this is a global variable
var2 = 60 # this is a global variable
def myfunction():
   "Change values of global variables"
   globals()['var1'] = globals()['var1']+10
   global var2
   var2 = var2 + 20

myfunction()
print ("var1:",var1, "var2:",var2) #shows global variables with changed values
```

On executing the code, it will produce the following output –

```
var1: 60 var2: 80
```

Example

Lastly, if you try to access a local variable in global scope, Python raises NameError as the variable in local scope can't be accessed outside it.

```
var1 = 50 # this is a global variable
var2 = 60 # this is a global variable
def myfunction(x, y):
   total = x+y
   print ("Total is a local variable: ", total)

myfunction(var1, var2)
print (total) # This gives NameError
```

It will produce the following error message –

```
Total is a local variable: 110
Traceback (most recent call last):
   File "C:\Users\user\examples\main.py", line 9, in <module>
   print (total) # This gives NameError
          ^^^^^
NameError: name 'total' is not defined
```

Business Analytics Certification

Java & Spring Boot Advanced Certification

Data Science Advanced Certification

Cloud Computing And DevOps

Advanced Certification In Business Analytics

Artificial Intelligence And Machine Learning

Chapters

Categories

Front-End Developer Certification

AWS Certification Training

Python Programming Certification

COMPILERS & EDITORS

Online Java Compiler

Online Python Compiler

Online Go Compiler

Online C Compiler

Online C++ Compiler

Online C# Compiler

Online PHP Compiler

Online MATLAB Compiler

Online Bash Terminal

Online SQL Compiler

Online Html Editor